PIPELINING PERFORMANCE

The 5 Stages: IF (Fetch), ID (Decode), EX (Execute), MEM (Memory), WB (Write-Back).

- **Latency:** Time for *one* instruction. Pipelining does NOT improve this.
- **Throughput:** Instructions per second. Pipelining vastly improves this.

Execution Time Math: Let N = Number of Instructions, k = Number of Pipeline Stages (usually 5), T_c = Clock Cycle Time.

- **Single-Cycle Time:** $= N \times T_c$
- **Pipelined Time (Ideal):** $= (k+N-1) \times T_{cpipe}$
- **Speedup:** $= \frac{Time_{single}}{Time_{pipelined}}$

If no hazards, instructions finish at a rate of 1 per cycle after the initial k cycles.

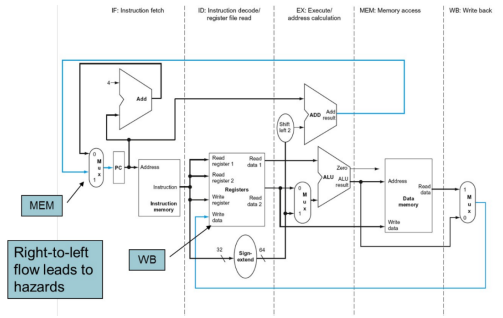
PIPELINE REGISTERS ("SUITCASES")

Registers inserted between stages to hold data & control signals for the current instruction.

- **IF/ID:** Holds PC and 32-bit Instruction.
- **ID/EX:** Holds Read Data 1 & 2, Sign-Ext Offset, and Control Signals (EX, MEM, WB).
- **EX/MEM:** Holds ALU Result, Data to Store, and Control Signals (MEM, WB).
- **MEM/WB:** Holds Memory Read Data, ALU Result, and Control Signals (WB).

The destination register (`Rd` or `Rt`) must be passed down the pipeline registers to WB so it doesn't write to the wrong register!

LEGv8 Pipelined Datapath



PIPELINE STAGE USAGE (WHO USES WHAT?)

- **R-Type (`ADD, SUB, AND`): IF, ID, EX, WB**
  - *Idle in MEM:* Math stays in CPU; doesn't access RAM.
- **Load (`LDUR`): IF, ID, EX, MEM, WB**
  - *Uses all 5 stages:* This is the critical path!
- **Store (`STUR`): IF, ID, EX, MEM**
  - *Idle in WB:* Writes to RAM, does NOT overwrite registers.
- **Cond. Branch (`CBZ`): IF, ID, EX**
  - *Idle in MEM, WB:* No RAM access, no register writes. ALU checks zero flag in EX.
- **Uncond. Branch (`B`): IF, ID**
  - *Idle in EX, MEM, WB:* Offset is shifted/added and PC is updated by the end of ID.

HAZARDS OVERVIEW

1. Structural Hazard: Hardware conflict (e.g., trying to read instructions and data from one memory simultaneously). *Fix:* Separate I-Mem and D-Mem (Caches).

2. Data Hazard: Instruction needs data before the previous instruction finishes writing it (RAW - Read After Write).

3. Control Hazard: Fetching next instruction depends on a branch outcome not yet calculated.

DATA HAZARDS & FORWARDING

Instead of stalling, pass the computed data directly to the ALU from the pipeline registers.

Forwarding Unit Logic (in EX stage)

Compares EX/MEM and MEM/WB destination registers with the ID/EX source registers.

1. EXE-Forwarding (1-Cycle Gap): *e.g., ADD X1... followed by SUB X2, X1...*

- `if (EX/MEM.RegWrite = 1` AND
- `EX/MEM.Rd == ID/EX.Rn)`
- → Forward from EX/MEM pipeline register.

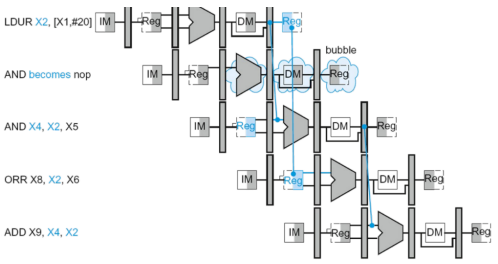
2. MEM-Forwarding (2-Cycle Gap): *e.g., ADD X1... followed by dummy, then SUB X2, X1...*

- `if (MEM/WB.RegWrite = 1` AND
- `MEM/WB.Rd == ID/EX.Rn)`
- → Forward from MEM/WB pipeline register.

Double Data Hazard Rule: If both hazards occur (e.g., two consecutive ADDs write to X1), the **most recent** instruction wins. EXE-forwarding takes priority!

LOAD-USE DATA HAZARD (STALLS)

You cannot forward backward in time! If an instruction needs data from a `LDUR` immediately, the data isn't retrieved from Memory until the end of MEM, but the ALU needs it at the start of EX.



Hazard Detection Unit (in ID stage)

Condition: `if (ID/EX.MemRead = 1` AND `(ID/EX.Rt == IF/ID.Rn1` OR `IF/ID.Rn2))`

Action (Insert 1 Bubble / Stall):

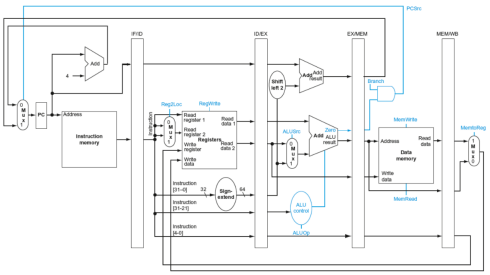
1. Force all control values in ID/EX to 0 (creates a NOP in the pipeline).
2. Prevent update of PC (fetches same instruction again).
3. Prevent update of IF/ID register (decodes same instruction again).

After 1 cycle stall, standard MEM-Forwarding resolves the hazard.

Code Scheduling: A compiler can fix Load-Use hazards by reordering instructions. Move an independent instruction between the Load and the Use to fill the delay slot without stalling.

CONTROL (BRANCH) HAZARDS

The Problem: If a branch is decided in the EX stage, the CPU has already fetched 3 wrong instructions if the branch is taken. **Hardware Fix:** Move target address adder and register comparator to the **ID stage**. This reduces the flush penalty from 3 cycles to **1 cycle**.



BRANCH PREDICTION

To avoid the 1-cycle penalty entirely, guess!

- **Static:** Always predict Taken (or Not Taken). Best for loops if you guess "Taken".
- **Dynamic:** Hardware records recent history in a Branch Prediction Buffer.

Dynamic Predictors

1-Bit Predictor:

- Guesses whatever happened last time.
- **Flaw:** Fails TWICE on nested loops. (Mispredicts at the loop exit, then mispredicts on the first iteration of the next run).

2-Bit Predictor (FSM):

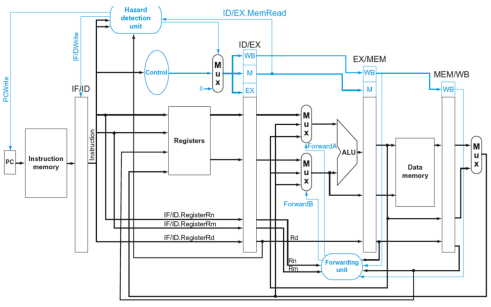
- 4 States: Strongly Taken (00), Weakly Taken (01), Weakly Not Taken (10), Strongly Not Taken (11).
- Must guess wrong **twice** in a row to flip its core prediction. Solves the loop exit problem.

Prediction Accuracy Math:

- Map sequence (e.g., T, T, N, T).
- *Accuracy* = (Correct Guesses) / (Total Guesses).

INSTRUCTION-LEVEL PARALLELISM (ILP)

- **Deeper Pipeline:** Less work per stage $\Longrightarrow$ Shorter clock cycle.
- **Multiple Issue (Superscalar):** Replicate pipeline stages to start multiple instructions per clock cycle.
- **IPC (Instructions Per Cycle):** Used instead of CPI when CPI ¡ 1. (e.g., A 4-way multiple-issue processor has a peak IPC of 4, or CPI of 0.25).



MEMORY HIERARCHY & LOCALITY

The Goal: Present the user with as much memory as is available in the cheapest technology (Disk) at the speed offered by the fastest technology (SRAM).

- **SRAM (Static RAM):** Cache. Fast (0.5ns), expensive, flip-flop based.
- **DRAM (Dynamic RAM):** Main Memory. Slower (50ns), cheaper, capacitor based (needs refreshing).
- **Flash/Disk:** Non-volatile, very slow, cheapest.

Principle of Locality: Programs access a small proportion of their address space at any time.

- **Temporal (Time):** If an item is accessed, it will likely be accessed again soon (e.g., loops).
- **Spatial (Space):** If an item is accessed, items whose addresses are close by will likely be accessed soon (e.g., arrays). Block transfers exploit this.

CACHE ORGANIZATIONS

1. Direct Mapped: Each memory block maps to exactly ONE cache slot.

- *Block Addr:* $\left\lfloor \frac{ByteAddr}{BytesPerBlock} \right\rfloor$
- *Location:* `Block Addr % (# Blocks in Cache)`.
- *Pros/Cons:* Fast and simple, but high conflict miss rate.

2. Fully Associative: A block can be placed ANYWHERE.

- *Location:* No index. Compare tags of all slots simultaneously.
- *Pros/Cons:* Lowest miss rate, but hardware is very expensive (needs a comparator for every entry).

3. N-Way Set Associative: Cache is divided into sets. Each set contains N blocks. A block maps to a specific Set, but can go in any of the N slots within that Set.

- *Location:* `Block Addr % (# Sets in Cache)`.

Replacement Policies (When a Set is Full):

- **LRU (Least Recently Used):** Replace the block untouched for the longest time. Hard to track for $>$ 4-way.
- **Random:** Evict a random block. Almost as good as LRU for high associativity, much cheaper to build.

CACHE TRACING TABLE SETUP (4-BLOCK CACHE)

Warning: If given BYTE addresses, divide by 4 first to get Word Addresses. Only use Word Addresses to find the Index and Tag!

1. Direct Mapped Organization

- **Index:** 2 bits (Start from right) $\rightarrow$ 4 slots.
- **Tag:** 1 bit (Start from left).

Addr	Tag	Index	H/M	Cache Content after access			
				00	01	10	11
010	0	10	M	—	—	M[010]	—

2. 2-Way Set Associative

- **Index:** 1 bit (Start from right) $\rightarrow$ 2 Sets.
- **Tag:** 2 bits (Start from left).
- *Rule:* Each Set holds 2 blocks. Update LRU on every hit/miss!

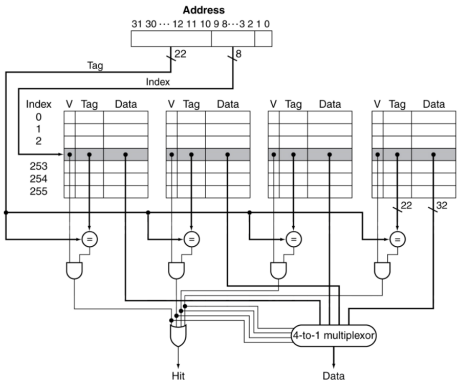
Addr	Tag	Index	H/M	Cache Content after access	
				Set 0 (Index 0)	Set 1 (Index 1)
010	01	0	M	M[010], —	—, —

3. Fully Associative

- **Index:** 0 bits (No index).
- **Tag:** 3 bits (The entire address is the tag).
- *Rule:* 1 Giant Set. Write list from MRU (Left) to LRU (Right).

Addr	Tag	H/M	Cache Content (Fully Associative)
010	010	M	M[010], —, —, —

Set Associative Cache Organization



Cache Address Subdivision (HW6 Math)

Given a 32-bit address, the CPU chops it into fields:
`[ Tag ] [ Index ] [ Word Offset ] [ Byte Offset ]`

1. Byte Offset: Which byte in the word?

- $\log_2$(Bytes per Word).
- *If 1 Word = 4 Bytes, Byte Offset is **ALWAYS 2 bits**.*

2. Word Offset: Which word in the block?

- $\log_2$(Words per Block).
- *If 1 Word/Block, offset = 0 bits. If 4 Words/Block, offset = 2 bits. If 8 words, offset = 3 bits.*

3. Index: Which Set does it go to?

- First, calculate Total Blocks: $\frac{\text{Total Capacity (Words)}}{\text{Words per Block}}$.
- Calculate Total Sets: $\frac{\text{Total Blocks}}{\text{N-Way Associativity}}$.
- *(For Direct Mapped, Associativity = 1).*
- Index Bits = $\log_2$(Total Sets).

4. Tag: The unique ID of the block.

- Tag Bits = 32 − (Index + Word Offset + Byte Offset).

HW6 Trap: If a problem states memory is strictly **Word-Addressable** (like HW6 Q2), there is NO Byte Offset and NO Word Offset. The entire address is just `[ Tag ] [ Index ]`.

HANDLING CACHE WRITES

When CPU executes `STUR`, it changes data. Cache and RAM are now inconsistent.

- **Write-Through:** Update BOTH cache and Main Memory immediately.
  - *Problem:* Extremely slow. Memory takes 100 cycles.
  - *Solution:* Use a **Write Buffer**. CPU writes to buffer and keeps going. Buffer writes to RAM in background.
- **Write-Back:** Update ONLY the cache.
  - Adds a **Dirty Bit** to each block. (1 = modified, 0 = clean).
  - The block is written back to Main Memory ONLY when it gets evicted/replaced. Fast, but complex.

Write Miss Policies (When data isn't in cache):

- **Write Allocate:** Fetch block into cache, then write. (Used with Write-Back).
- **Write Around (No-Allocate):** Don't fetch the block. Write directly to RAM. (Used with Write-Through).

Cache Performance Equations

Average Memory Access Time (AMAT):

$$\text{AMAT} = \text{Hit Time} + (\text{Miss Rate} \times \text{Miss Penalty})$$

Example: 1ns clock, Hit = 1 cycle, Miss Penalty = 20 cycles, Miss Rate = 5%. AMAT = $1 + (0.05 \times 20) = 2$ cycles.

Memory Stall Cycles:

$$\text{Stalls} = \text{Instrs} \times \frac{\text{Misses}}{\text{Instr}} \times \text{Miss Penalty}$$

Effective CPU Time:

$$\text{Time} = (\text{Execution Cycles} + \text{Stall Cycles}) \times T_c$$

MULTILEVEL CACHES

One cache isn't enough.

- **L1 Cache (Primary):** Attached to CPU. Focuses on **minimal hit time** to match CPU clock speed. Usually smaller, lower associativity.
- **L2 Cache:** Services L1 misses. Focuses on **low miss rate** to avoid hitting slow Main Memory. Usually larger, higher associativity.

Multilevel AMAT Math: If L1 misses, it goes to L2. If L2 misses, it goes to RAM. The "Miss Penalty" of L1 is actually the AMAT of L2!

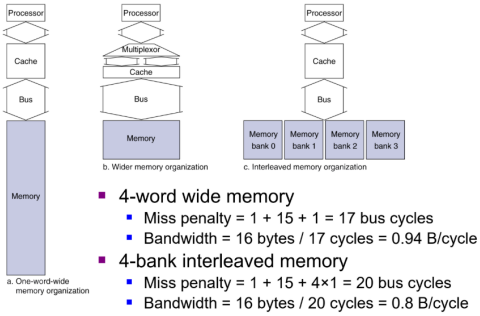
$$\text{Penalty}_{L1} = \text{Hit}_{L2} + (\text{MissRate}_{L2} \times \text{RAM Penalty})$$

$$\text{Overall AMAT} = \text{Hit}_{L1} + (\text{MissRate}_{L1} \times \text{Penalty}_{L1})$$

IMPROVING MEMORY BANDWIDTH

If Cache misses, fetching from DRAM is slow.

- **Wide Memory:** Increase bus width (fetch 4 words at once). Pays the setup latency only once per chunk.
- **Interleaved Memory:** Organize RAM into Banks. Send addresses to multiple banks simultaneously.



CACHE VALID BITS & TAGS

- **Valid Bit:** 1 bit per block. Initializes to 0 (garbage data). Becomes 1 when real data is loaded. CPU ignores blocks with Valid = 0.
- **Tag Comparison:** On an access, CPU extracts the Index to find the Set. It then compares the Address Tag to ALL Tags in that Set simultaneously. If Tag matches AND Valid=1, it is a **HIT**.